

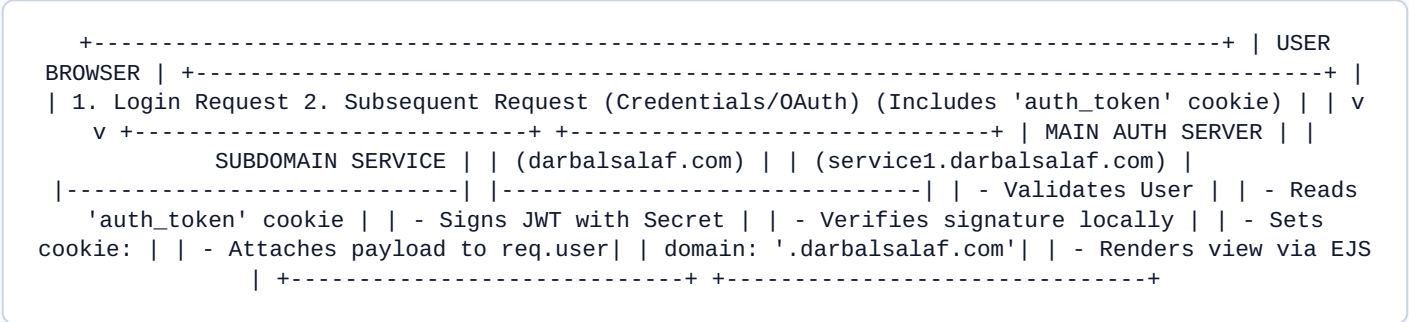
Express.js Cross-Subdomain Authentication Guide

Unified JWT Cookie Architecture across Independent Express Services

PRIMARY DOMAIN	AUTHENTICATION MECHANISM	FRAMEWORK & VIEW ENGINE
darbalsalaf.com	JWT + Shared HttpOnly Cookie	Express.js & EJS

1. Architectural Overview

When serving multiple independent services across subdomains (e.g., `service1.darbalsalaf.com`, `service2.darbalsalaf.com`), user authentication must be seamless. This architecture utilizes **Domain-Scoped JWT Cookies** to allow a single sign-on (SSO) experience across all services without requiring a centralized session store database.



Key Principle: Cookie Scope

By setting the cookie parameter `domain: '.darbalsalaf.com'` (with a leading dot), modern Web Browsers automatically transmit the `auth_token` cookie on every HTTP request sent to any subdomain of `darbalsalaf.com`.

2. Main Authentication Server (Router Code)

In your main authentication router (`routes/account.js`), helper functions and routes issue the signed JWT cookie upon successful user verification, password creation, or OAuth authentication (Google/GitHub).

```
// routes/account.js excerpt
const jwt = require("jsonwebtoken");

const JWT_SECRET = process.env.JWT_SECRET || "your_fallback_super_secret_key";
const COOKIE_DOMAIN = process.env.COOKIE_DOMAIN || ".darbalsalaf.com";

function setAuthCookie(res, userData) {
  const token = jwt.sign(
    {
      email: userData.email,
      name: userData.name,
      image: userData.image || "",
    },
    JWT_SECRET,
    { expiresIn: "30d" }
  );
```

```

res.cookie("auth_token", token, {
  domain: COOKIE_DOMAIN, // Shares cookie with *.darbalsalaf.com
  httpOnly: true,         // Prevents client-side JS XSS access
  secure: process.env.NODE_ENV === "production",
  sameSite: "lax",
  maxAge: 1000 * 60 * 60 * 24 * 30, // 30 days
});
}

```

3. Subdomain Authentication Middleware

In every separate Express project running on a subdomain, install `cookie-parser` and `jsonwebtoken`. Create an authentication middleware to verify the token locally.

```

// middleware/auth.js (Subdomain Project)
const jwt = require("jsonwebtoken");
const JWT_SECRET = process.env.JWT_SECRET || "your_fallback_super_secret_key";

function verifyUser(req, res, next) {
  const token = req.cookies.auth_token;

  if (!token) {
    req.user = null;
    return next();
  }

  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    req.user = decoded; // Contains { email, name, image }
  } catch (err) {
    req.user = null;
  }
  next();
}

function requireAuth(req, res, next) {
  if (!req.user) {
    return res.redirect("https://darbalsalaf.com/login");
  }
  next();
}

module.exports = { verifyUser, requireAuth };

```

4. Passing & Rendering User Data in EJS

To make user data globally available across all EJS templates without passing `{ user: req.user }` in every single route handler, attach `req.user` to Express's `res.locals`.

```

// server.js (Subdomain Project)
const express = require("express");
const cookieParser = require("cookie-parser");
const { verifyUser, requireAuth } = require("../middleware/auth");

```

```
const app = express();
app.use(cookieParser());
app.use(verifyUser);

// Make user data accessible to ALL EJS templates automatically
app.use((req, res, next) => {
  res.locals.user = req.user;
  next();
});

// Protected Service Route
app.get("/dashboard", requireAuth, (req, res) => {
  res.render("dashboard.ejs"); // 'user' is already available in EJS
});
```

EJS Template Example (`views/dashboard.ejs`)

```
<% if (user) { %>
  <div class="user-card">
    <% if (user.image) { %>
      
    <% } %>
    <h3>Welcome back, <%= user.name %>!</h3>
    <p>Email: <%= user.email %></p>
    <a href="https://darbalsalaf.com/logout">Logout</a>
  </div>
<% } else { %>
  <a href="https://darbalsalaf.com/login">Please Log In</a>
<% } %>
```

5. Local Development Configuration

Browsers ignore domain-scoped cookies on raw `localhost`. To test multi-subdomain cookie sharing locally on your machine, edit your operating system's hosts file.

OS / Location	Configuration Line in Hosts File
Windows: <code>C:\Windows\System32\drivers\etc\hosts</code>	<code>127.0.0.1 darbalsalaf.local</code> <code>service1.darbalsalaf.local</code>
Linux/macOS: <code>/etc/hosts</code>	<code>127.0.0.1 darbalsalaf.local</code> <code>service1.darbalsalaf.local</code>

In your local `.env` file, update the domain variable to match:

```
COOKIE_DOMAIN=.darbalsalaf.local
JWT_SECRET=your_super_secret_key
```

6. Summary Checklist

- ✓ **JWT Secret Consistency:** Ensure all subdomain projects share the exact same `JWT_SECRET` string in their environment settings.
- ✓ **HttpOnly Security:** Ensure `httpOnly: true` is enabled on cookies to prevent

cross-site scripting (XSS) attacks. ✓ **CORS Settings:** If using client-side AJAX/fetch between subdomains, enable `credentials: true` in CORS configurations. ✓ **HTTPS Enforcement:** Enable `secure: true` for cookies in production so tokens are transmitted strictly over encrypted HTTPS connections.